Orleans University

Polytech Engineering school

AL & Big Data Lab Report

Prepared by:

Vincent Mwenda Mworia

Instructor:

Prof. Frédéric Ros

Date:

1st December 2025

Academic Year 2025/2026

Table of Contents

GITHUB REPOS FOR THE CODE?

References

1. University of Orleans. (2025). Machine Learning Lab Manual Series (TPs): K-Means, DBSCAN, Hierarchical Clustering, Regression, KNN, Logistic Regression, Gradient Descent, and Perceptron. Department of Computer Science. Prepared by Prof. Prof. Frédéric Ros
2. McKinney, W. (2022). Python for Data Analysis. O'Reilly Media.

Lab 1 - K-Means Clustering

Objective

This experiment aims to explore how the K-Means clustering algorithm groups data into clusters based on similarity.

The lab focuses on:

- Understanding the K-Means process.
- Applying it to a simple 2D dataset.
- Finding the best number of clusters using the Elbow method.
- Testing it on a real-world dataset (Iris).
- Observing its strengths and weaknesses.

Theory

K-Means is an unsupervised learning algorithm used to discover natural groupings (clusters) in data. It partitions the data into K clusters such that points within a cluster are more similar to each other than to those in other clusters.

The algorithm follows these main steps:

1. **Start:** Choose *K* random points as initial centroids.
2. **Assign:** Allocate each data point to the nearest centroid.
3. **Update:** Recalculate centroids as the average of all points in each cluster.
4. **Repeat:** Continue until cluster centers no longer move significantly.

The algorithm minimizes the **sum of squared distances** between each point and its assigned centroid (called *inertia*).

Code

The K-Means clustering code can be found at the following GitHub link: K-Means_code

Implementation on Synthetic Data

To visualize how K-Means behaves, a simple 2D dataset with three clusters was created using make_blobs().

The model successfully identified three distinct clusters in the dataset. The centroids, represented by black “X” symbols, indicate the center of each cluster. Each color in the plot corresponds to a separate cluster, clearly showing separation between the data groups and confirming that the K-Means algorithm effectively grouped similar data points together.

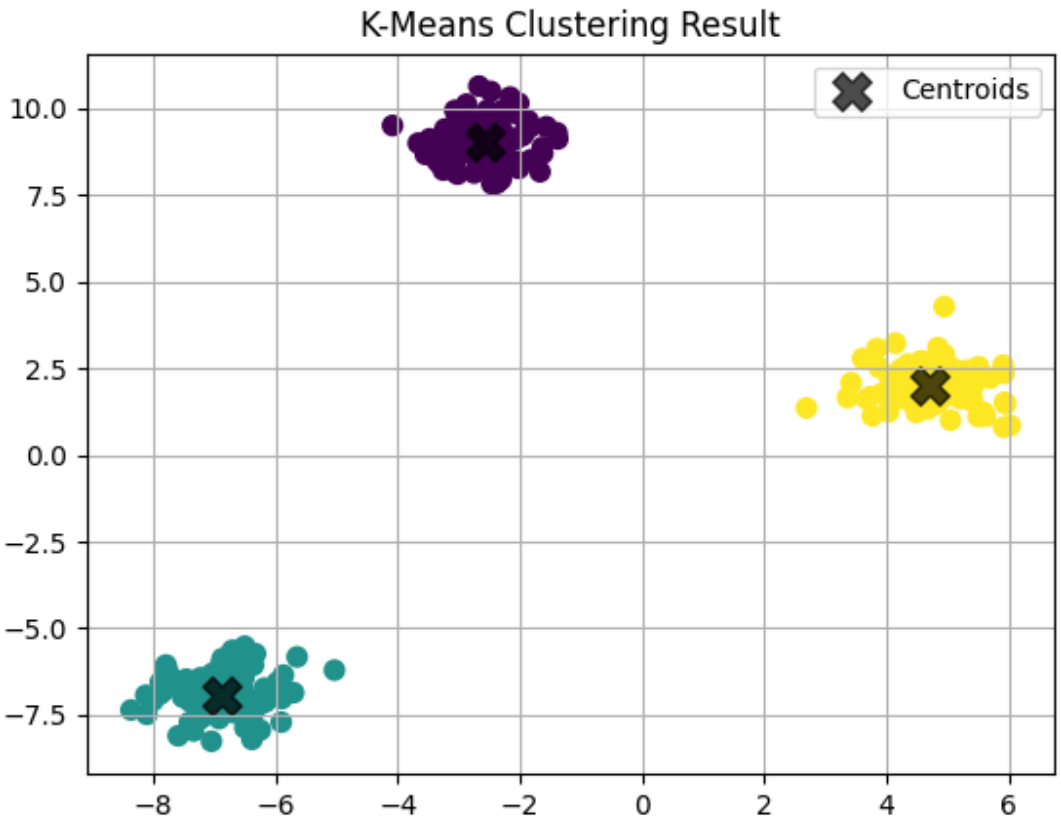


Figure 1. *K-Means clustering result on a 2D synthetic dataset.*

Determining the Optimal Number of Clusters

The Elbow Method was used to determine the most appropriate value for K. It measures how the total inertia (within-cluster sum of squares) changes as K increases.

As K grows, inertia naturally decreases because clusters become smaller and data points move closer to their respective centroids. However, beyond a certain point, the improvement becomes minimal hence additional clusters start capturing random noise rather than meaningful structure. Moreover, using too many clusters can lead to overfitting, where the model fits the data too precisely and loses generalization ability.

Therefore, the optimal value of K is chosen at the “elbow point” where adding more clusters no longer results in a significant decrease in inertia. This choice balances accuracy with simplicity, preventing the model from overfitting.

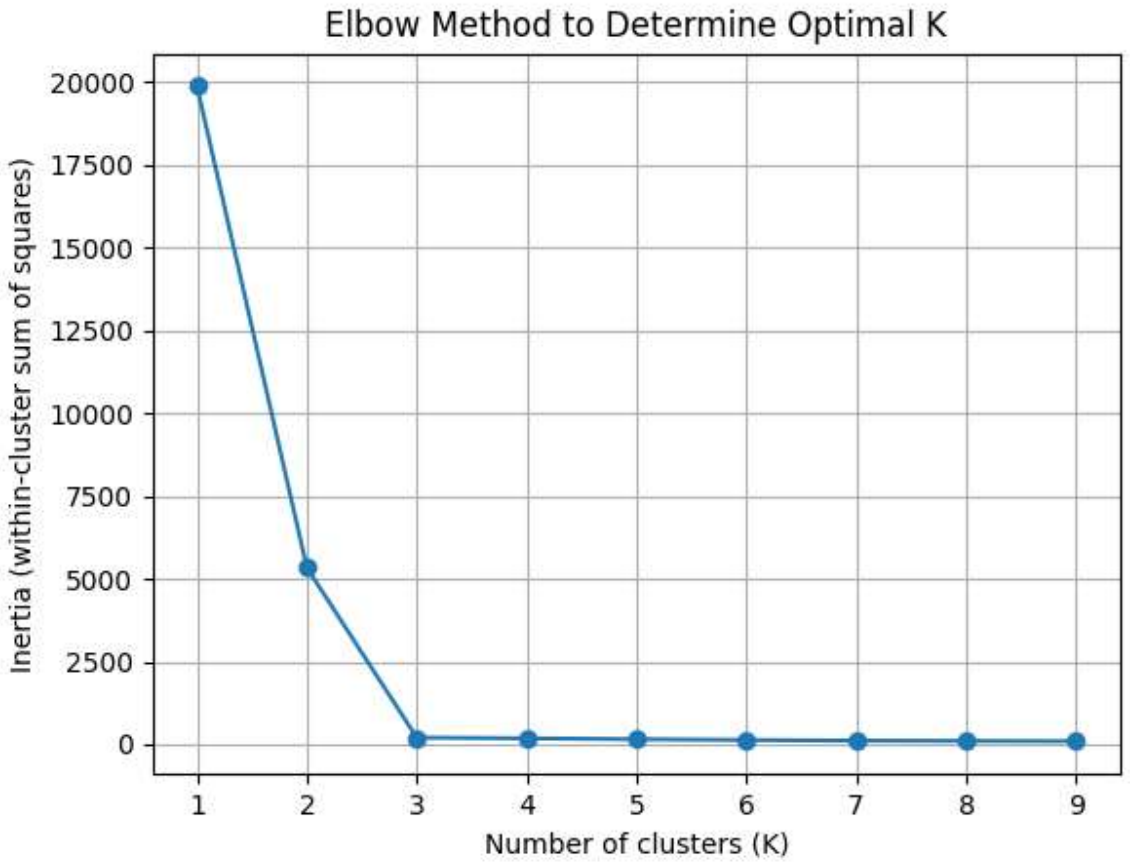


Figure 2. *Elbow Method used to determine the optimal number of clusters.*

Applying K-Means to the Iris Dataset

The algorithm was tested on the Iris flower dataset, which contains 150 samples and four numerical features.

Before clustering, the data was standardized using StandardScaler() to ensure that all features contributed equally to the distance calculations. This transformation scales each feature to have a mean of 0 and a standard deviation of 1, making the variables comparable across dimensions.

Principal Component Analysis (PCA) was also applied to reduce the data from four dimensions to two. This dimensionality reduction technique preserves most of the variance in the dataset while enabling effective visualization of the clustering results in a 2D plot.

K-Means successfully identified three major groups, roughly corresponding to the three Iris species. However, some overlap occurred between the purple cluster (versicolor) and the yellow cluster (virginica), indicating that the K-Means algorithm can struggle when clusters overlap or are not linearly separable in feature space.

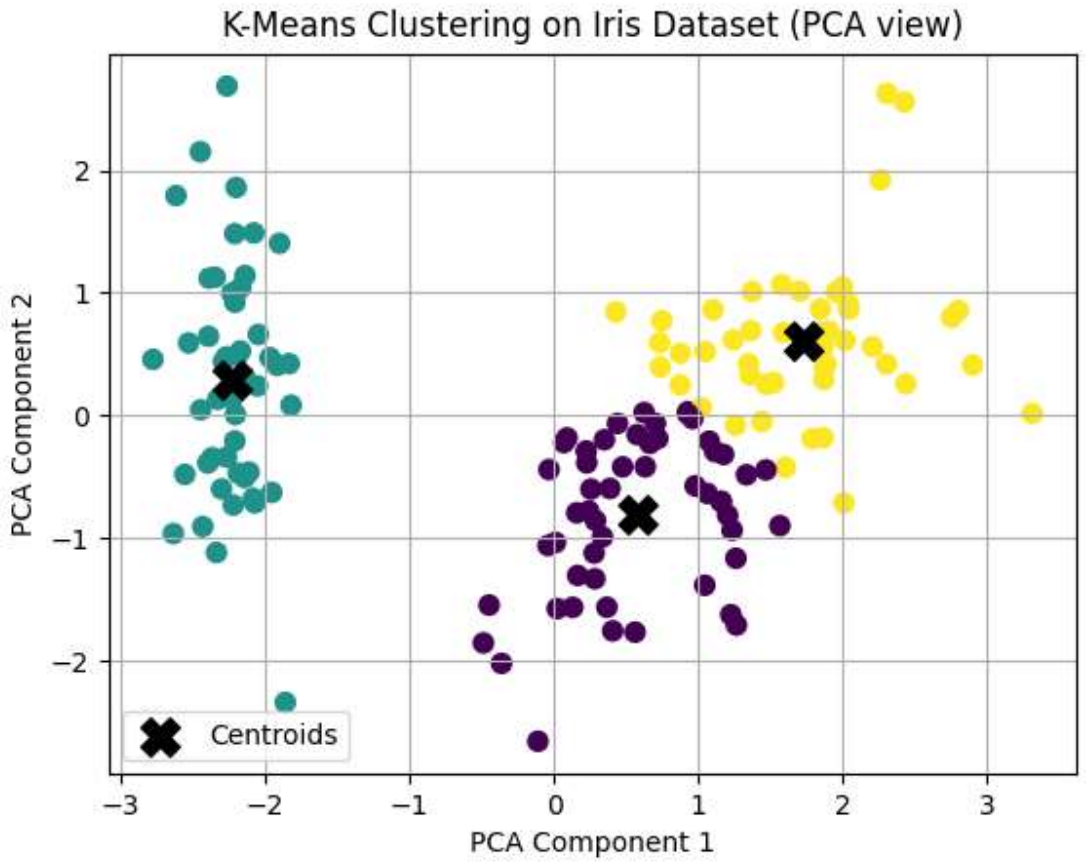


Figure 3. *K-Means clustering on the Iris dataset (2D PCA projection).*

Exploring Limitations with Non-Spherical Data

To examine how K-Means performs on irregularly shaped datasets, two synthetic datasets were generated. One was shaped like interlocking moons and another like concentric circles.

In both cases, the algorithm failed to correctly separate the curved clusters because K-Means relies on Euclidean distance to assign points to the nearest centroid. As a result, it naturally forms circular cluster boundaries. This assumption works well for spherical data but performs poorly when clusters are non-linear or non-spherical. Consequently, K-Means misclassifies several data points in regions where the clusters bend or overlap.

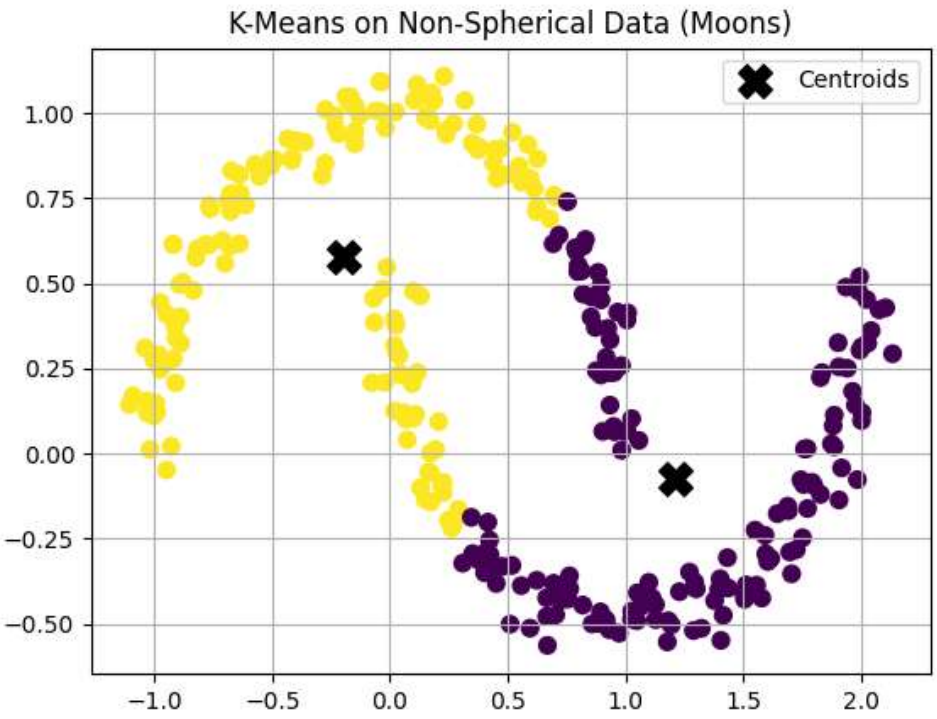


Figure 4. *K-Means clustering on non-spherical data (two interlocking moons)*

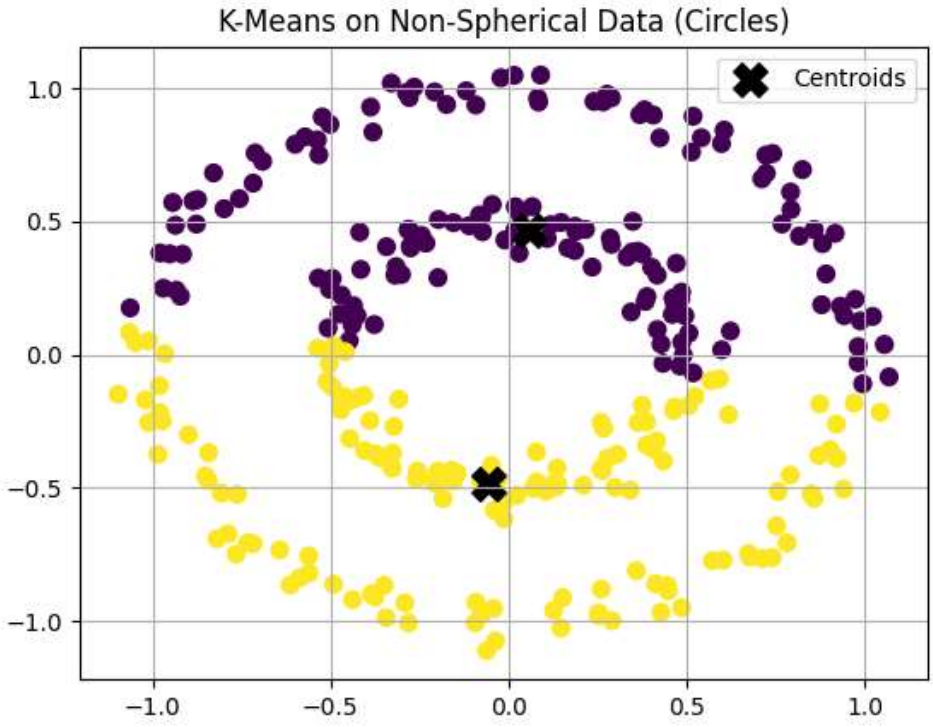


Figure 5. *K-Means clustering on non-spherical data (concentric circles)*

Cluster Evaluation (Silhouette Score)

The Silhouette Score measures how well each data point fits within its assigned cluster compared to other clusters. The score ranges from -1 to 1, where a value closer to 1 indicates that the data points are well clustered, while lower or negative values suggest poor separation or incorrect clustering. The highest silhouette value was observed at K = 2, indicating that the data is most cleanly divided into two distinct clusters. However, K = 3 still provides a reasonable structure that aligns with the known Iris species groupings.

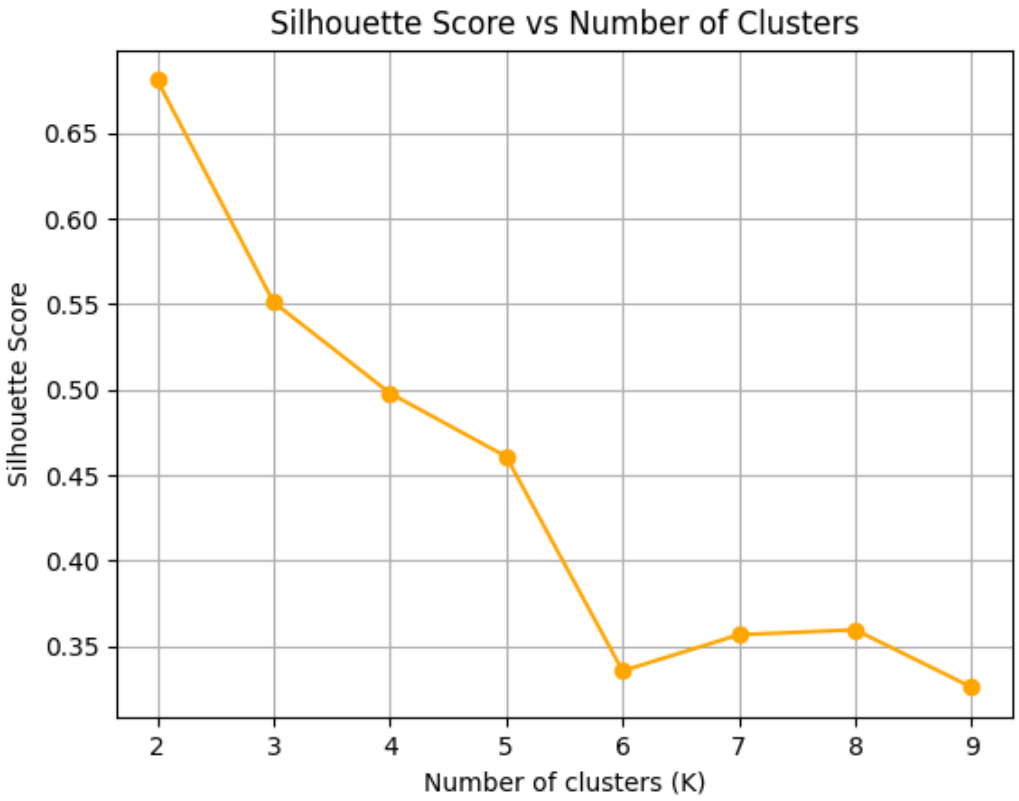


Figure 6. *Silhouette Score versus number of clusters for the Iris dataset.*

Questions for Discussion

1. What are the limitations of K-Means?

   K-Means requires specifying K in advance, assumes spherical and equally sized clusters, and performs poorly on irregular, overlapping, or outlier-heavy data.

2. What happens if clusters are not spherical?

   K-Means forces circular boundaries, so it struggles with curved or uneven clusters, often splitting points from the same true group or grouping distant points together incorrectly.

3. Does K-Means always find the best result?

   No. K-Means can get stuck in a local minimum, meaning it may find a suboptimal solution depending on where the centroids were initially placed.

4. What if centroids are initialized poorly?

   Poor initialization can lead to unbalanced or incorrect clusters and may also cause slower convergence. The K-Means++ method reduces this issue by choosing well-spaced initial centroids for more stable and accurate results.

Conclusion

K-Means is one of the most widely used and intuitive algorithms for identifying structure in data. It performs well on compact, well-separated clusters but struggles with irregular, overlapping, or non-spherical shapes. Overall, it serves as an excellent starting point for clustering analysis, though more advanced algorithms such as DBSCAN or Gaussian Mixture Models (GMMs) are better suited for complex or non-linear datasets.

K-Means is most commonly applied in customer segmentation, image compression, document grouping, and market analysis, where cluster boundaries are approximately convex and the number of groups can be estimated beforehand.

Lab 2 - DBSCAN Clustering

Objective

This experiment aims to explore how the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm discovers clusters by local density and identifies outliers. The lab focuses on:

- Understanding DBSCAN's core ideas and parameters (**eps**, **min_samples**) and the roles of core, border, and noise points.
- Applying DBSCAN to a non-spherical 2D dataset (two moons) and examining parameter sensitivity.
- Comparing DBSCAN with K-Means on the same data to highlight shape and outlier handling differences.
- Running DBSCAN on the Iris dataset (with PCA for visualization) and counting clusters vs. noise points.
- Noting practical limitations (single global **eps**, varying densities) and when alternatives like HDBSCAN may be preferable.

Theory

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups together points closely packed in space, while marking isolated points as noise. It defines clusters based on two parameters:

- ε (eps): the neighborhood radius around a point.
- min_samples: the minimum number of points required to form a dense region.

Conceptually, the method considers a neighborhood of radius ε around each point in the dataset. If this neighborhood contains at least min_samples points, the point is classified as a **core point**. Points that lie within the ε-neighborhood of a core point but do not themselves meet the density requirement are referred to as **border points**. Points that are neither core nor border points are designated as **noise** or **outliers**.

Clusters are then constructed by connecting core points whose neighborhoods overlap, thereby forming continuous regions of high density. Unlike K-Means, DBSCAN does not require specifying the number of clusters and can identify clusters of arbitrary shapes, making it robust for non-spherical data and outlier detection. However, its performance depends strongly on suitable ε and min_samples values.

Code

The DBSCAN clustering code can be found at the following GitHub link: DBSCAN_code

Implementation on Synthetic Non-Spherical Data

A two-dimensional non-spherical dataset was generated using the make_moons() function with 300 samples and a noise level of 0.05. This dataset was selected because it cannot be effectively separated by algorithms that assume spherical clusters, such as K-Means. The DBSCAN algorithm was applied with parameters ε = 0.2 and min_samples = 5 to identify the two crescent-shaped clusters.

These parameter values were selected empirically through visual inspection of the distance distribution, as they provided a balance between accurately detecting the two main clusters and minimizing the number of points classified as noise.

A smaller ε value tends to fragment the data into numerous small clusters, often increasing the number of noise points and resulting in underfitting, where the model is too rigid to capture the full cluster structure. Conversely, an excessively large ε may merge distinct regions into a single cluster, leading to overfitting, where the model becomes too general and fails to preserve meaningful boundaries. Similarly, a low min_samples value allows small or spurious clusters to form, while a high min_samples value enforces stricter density requirements that may exclude legitimate but less dense clusters.

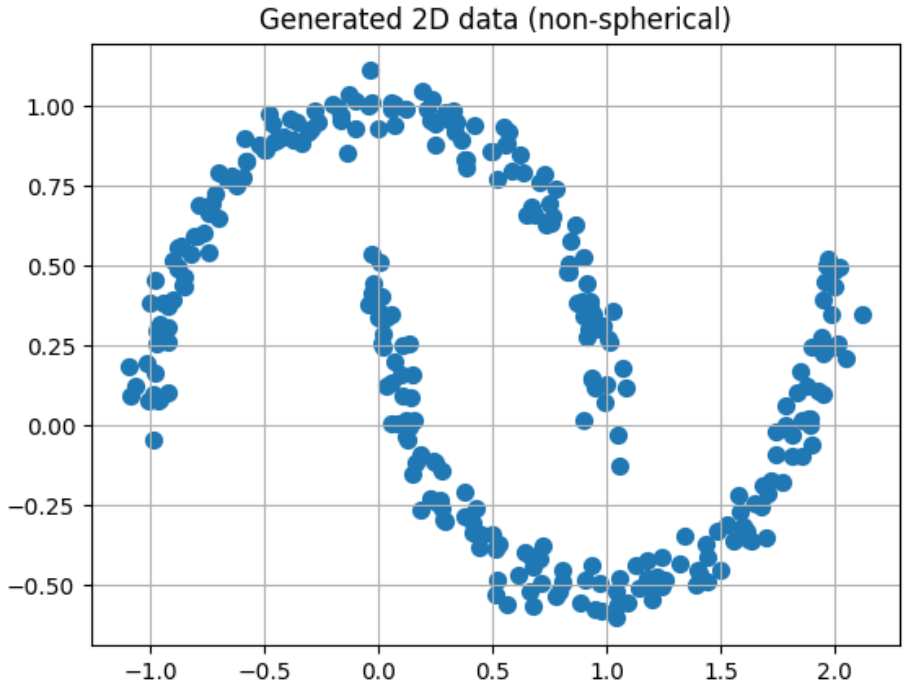


Figure 7. *Generated two-dimensional non-spherical dataset (two-moons) created using the make_moons() function.*

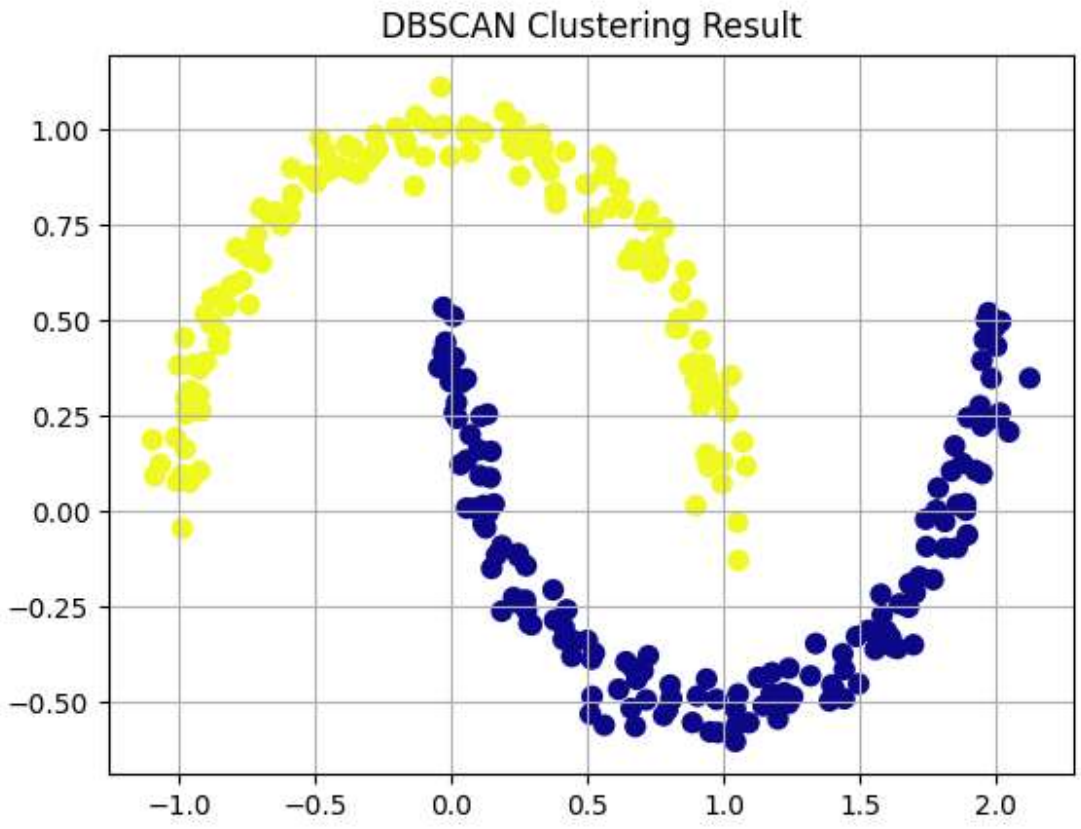


Figure 8. *DBSCAN clustering result on the two-moons dataset using parameters ε = 0.2 and min_samples = 5.*

Parameter Sensitivity

To examine the influence of DBSCAN's parameters, experiments were conducted by varying ε and min_samples independently.

As ε increased, the neighborhood radius expanded, causing the algorithm to transition from detecting multiple small, fragmented clusters (when ε was small) to merging the two crescent-shaped structures into a single cluster (when ε was large). Smaller ε values led to excessive fragmentation and noise (underfitting), while larger ε values caused overgeneralization and cluster merging (overfitting).

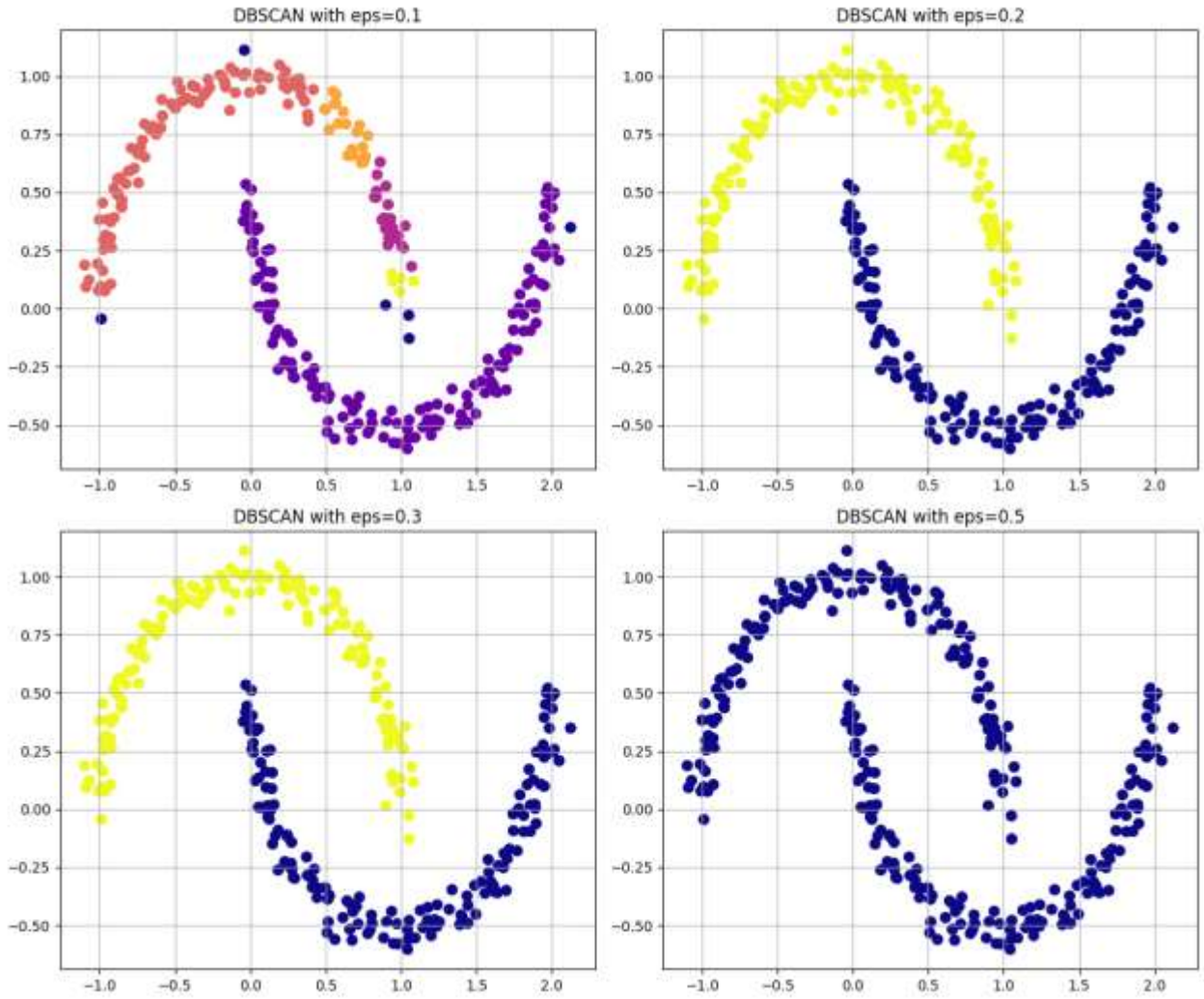


Figure 9. *DBSCAN results with varying ε values (0.1, 0.2, 0.3, 0.5) and fixed min_samples = 5.*

Similarly, increasing min_samples made the density criterion progressively stricter, reducing the number of core points and increasing noise, especially at higher values. Smaller min_samples values (e.g., 3–5) produced stable and coherent clusters, while very large values (e.g., 20) led to overly conservative clustering that excluded many valid points at the edges of each crescent.

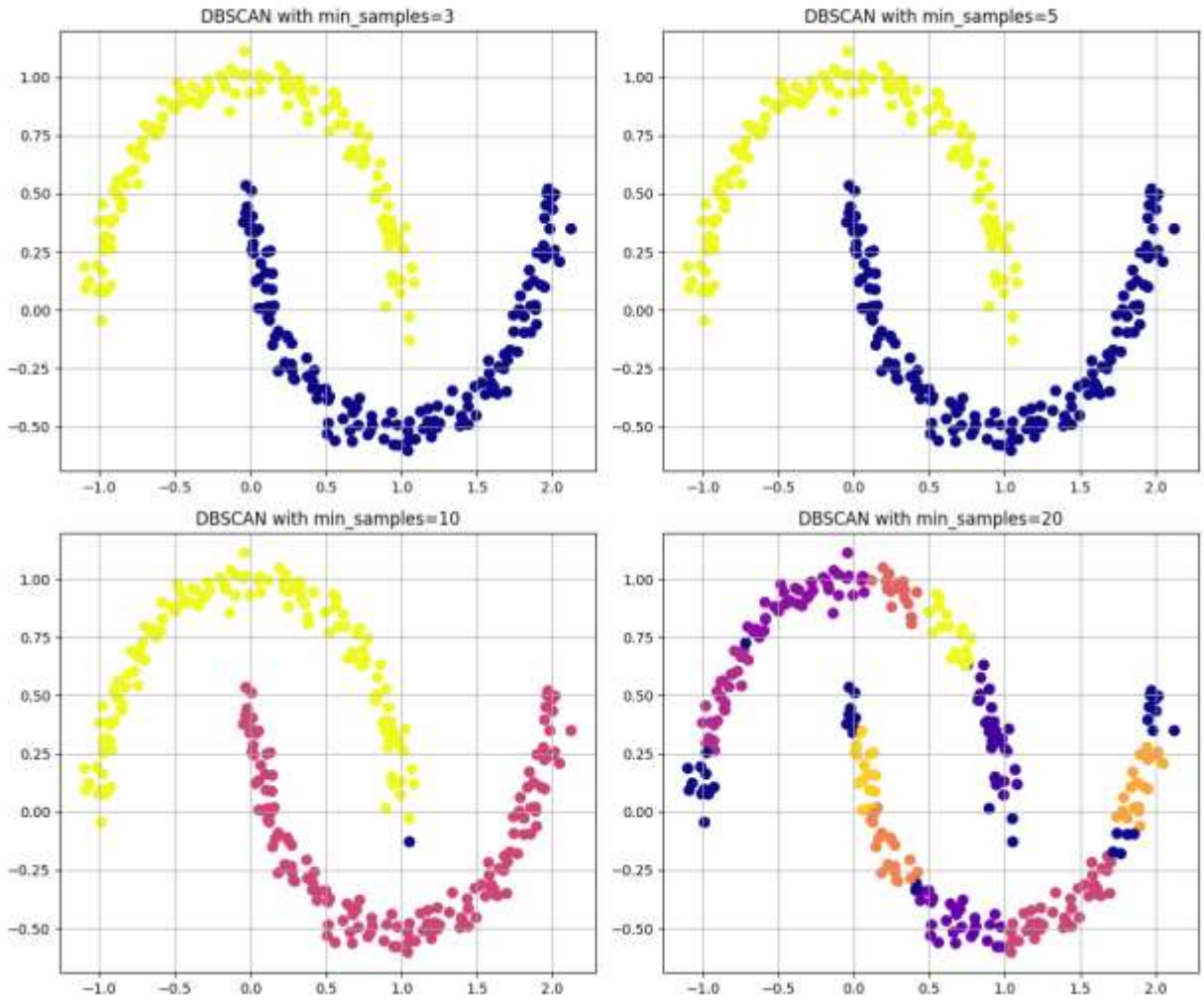


Figure 10. *DBSCAN results with varying min_samples values (3, 5, 10, 20) and fixed ε = 0.2.*

An intermediate configuration (ε = 0.2, min_samples = 5) yielded the most coherent and balanced clustering.

Comparing to K-Means

The same non-spherical two-moons dataset was also clustered using K-Means with k = 2. K-Means divided the data into two regions using straight, centroid-based boundaries that do not follow the curved structure of the dataset. As a result, several points near the junction of the two crescents were misclassified due to the algorithm's assumption of spherical cluster geometry and its reliance on Euclidean distance to fixed centroids.

DBSCAN, in contrast, accurately identified the two curved clusters without requiring the number of clusters in advance. Its density-based approach captured the true manifold of the data and correctly separated the clusters while labeling a few boundary points as noise. This result demonstrates DBSCAN's ability to model irregular and non-convex shapes that K-Means cannot effectively represent.

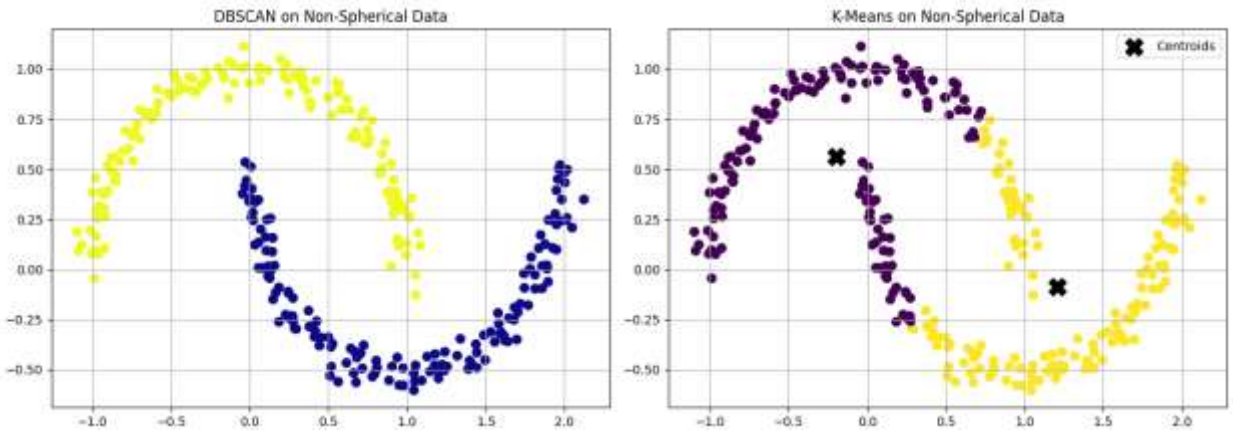


Figure 11. *Comparison of clustering performance on the non-spherical two-moons dataset between DBSCAN (ε = 0.2, min_samples = 5) on the left and K-Means (k = 2) on the right.*

DBSCAN on the Iris Dataset (Real-World Multidimensional Data)

The Iris dataset was used to evaluate DBSCAN on real-world, multi-dimensional data. The feature variables were standardized using StandardScaler to ensure equal contribution of all attributes to the distance metric. Dimensionality reduction was then performed using Principal Component Analysis (PCA) to project the four-dimensional data into two principal components for visualization purposes only.

DBSCAN was applied to the standardized data using parameters ε = 0.6 and min_samples = 5. The algorithm identified several compact regions of high density corresponding to potential clusters while labeling certain ambiguous samples as noise. The resulting two-dimensional PCA projection revealed visually separable groups that broadly corresponded to the Iris species, although some overlap occurred between Iris versicolor and Iris virginica, which are known to have similar feature distributions.

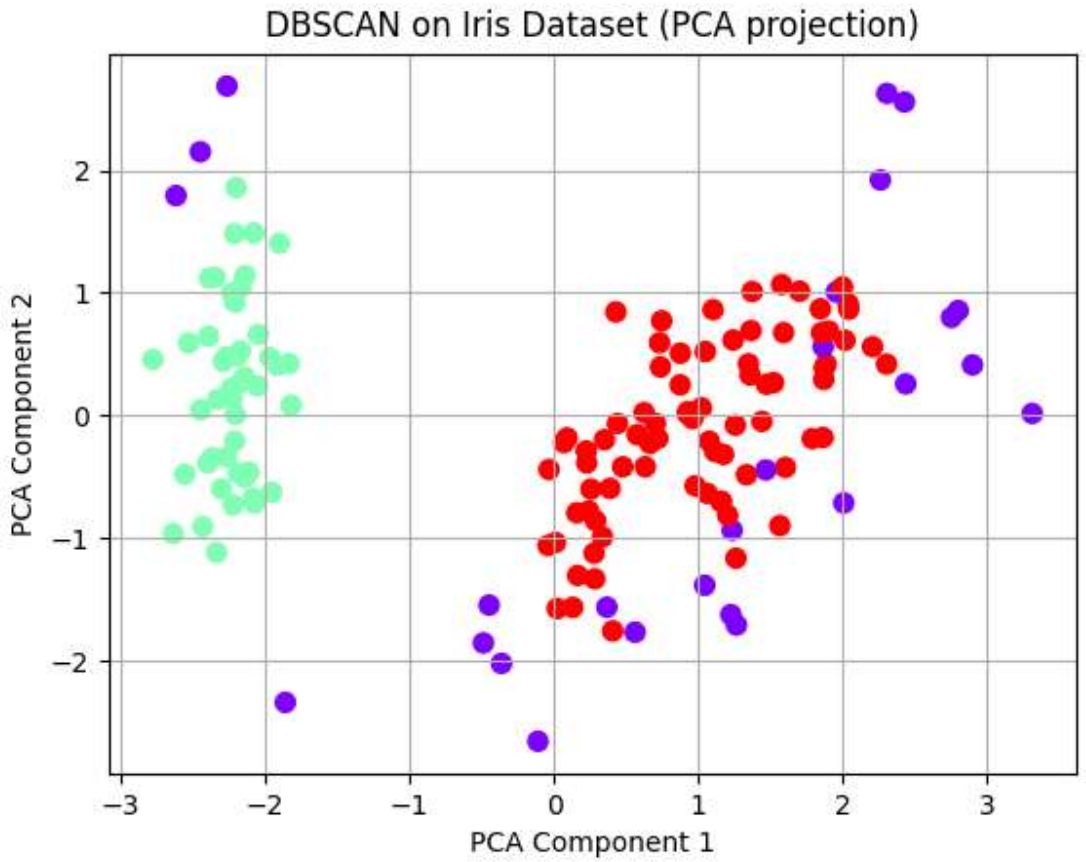


Figure 12: *DBSCAN clustering of the Iris dataset visualized after PCA projection (ε = 0.6, min_samples = 5).*

Data with Varying Density

To examine DBSCAN's behaviour when cluster densities differ, a synthetic dataset was generated using the make_blobs() function with cluster standard deviations of [0.5, 1.5, 0.3], creating three groups of unequal density. The algorithm was executed with parameters ε = 0.8 and min_samples = 5.

The resulting plot shows that DBSCAN correctly identifies the two denser clusters but partially merges or misclassifies points in the sparser, more diffuse cluster. This occurs because DBSCAN applies a single global density threshold (ε), which cannot simultaneously accommodate both dense and sparse regions. Consequently, points belonging to low-density clusters are either labeled as noise or absorbed into neighboring clusters.

This experiment highlights DBSCAN's sensitivity to varying density and motivates the use of adaptive approaches such as HDBSCAN, which can handle non-uniform density distributions by adjusting neighborhood thresholds locally.

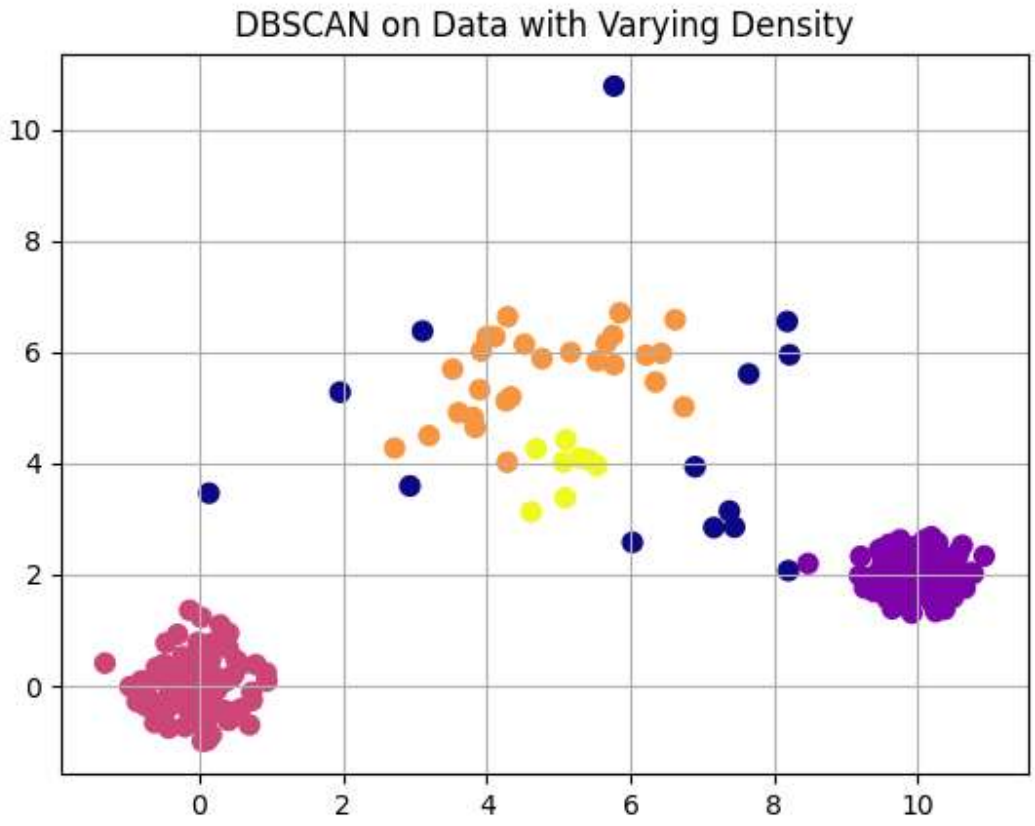


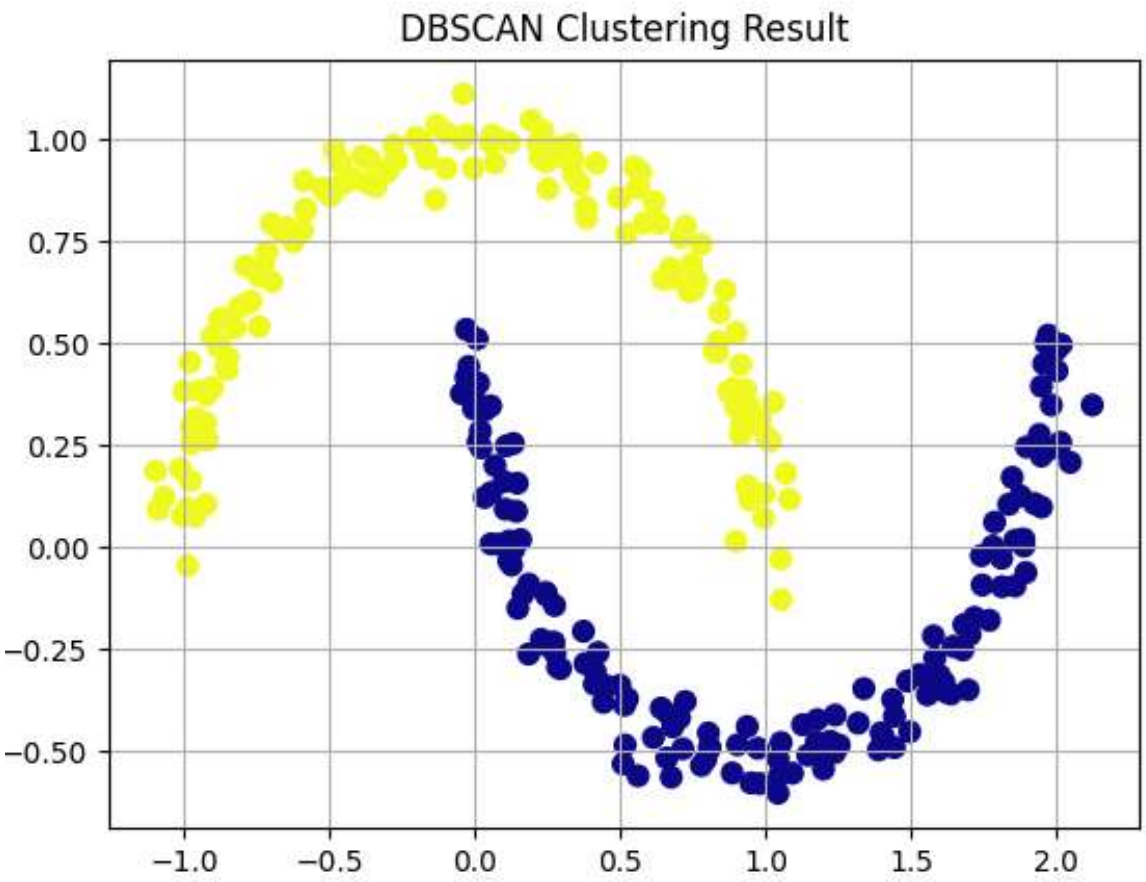
Figure 13: *DBSCAN clustering of the Iris dataset visualized after PCA projection (ε = 0.6, min_samples = 5).*

Using Silhouette Score and Adjusted Rand Index to Evaluate Clustering Quality

To quantitatively assess DBSCAN's performance, two standard clustering evaluation metrics were computed on the two-moons dataset: the Silhouette Score and the Adjusted Rand Index (ARI).

The Silhouette Score of 0.328 indicates moderate cluster separation. Points within each cluster are generally closer to one another than to points in other clusters. However, some points near the edges of the crescent-shaped regions lie close in Euclidean distance to points in the neighboring cluster. Because the Silhouette metric evaluates separation based purely on straight-line distances, it penalizes these boundary points even though they are correctly assigned. This results in a moderate score, which is expected for non-spherical data such as the two-moons structure.

The Adjusted Rand Index (ARI) achieved a perfect score of 1.000, confirming that DBSCAN's cluster assignments exactly matched the true underlying structure of the dataset. This result demonstrates DBSCAN's strength in identifying complex, non-spherical cluster shapes that are difficult for centroid-based algorithms such as K-Means to represent accurately.



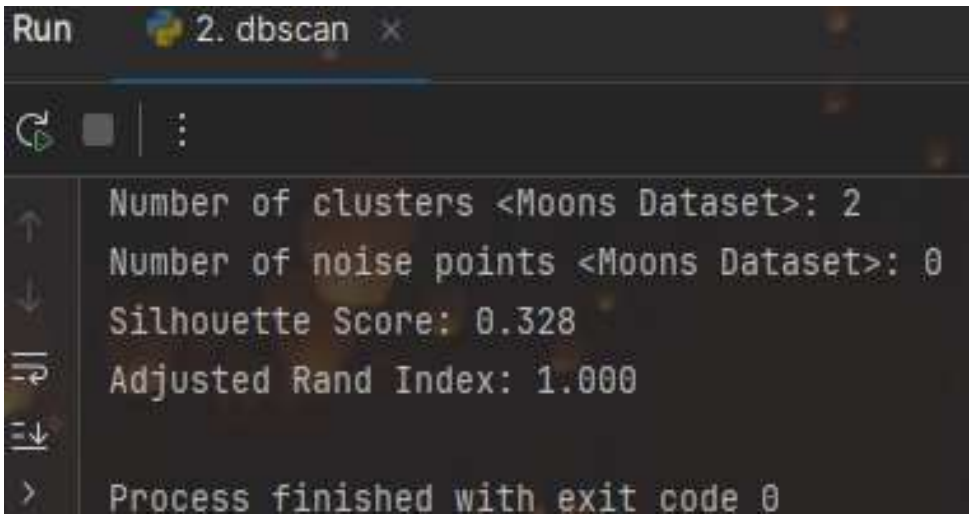


Figure 14: *Quantitative evaluation of DBSCAN performance on the two-moons dataset (ε = 0.2, min_samples = 5).*

HDBSCAN on Varying-Density Data

To address DBSCAN's limitation in handling clusters of unequal density, the same synthetic dataset with varying cluster standard deviations [0.5,1.5,0.3] was analyzed using HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise). Unlike DBSCAN, HDBSCAN does not rely on a single global ε value; instead, it builds a hierarchy of density levels and extracts stable clusters based on their persistence across scales.

With parameters min_cluster_size = 15 and min_samples = 5, HDBSCAN successfully detected three main clusters and a small number of noise points. Compared to DBSCAN, which struggled to separate the sparser middle group, HDBSCAN adaptively adjusted to local density variations, resulting in more accurate cluster boundaries and reduced misclassification of low-density points as noise.

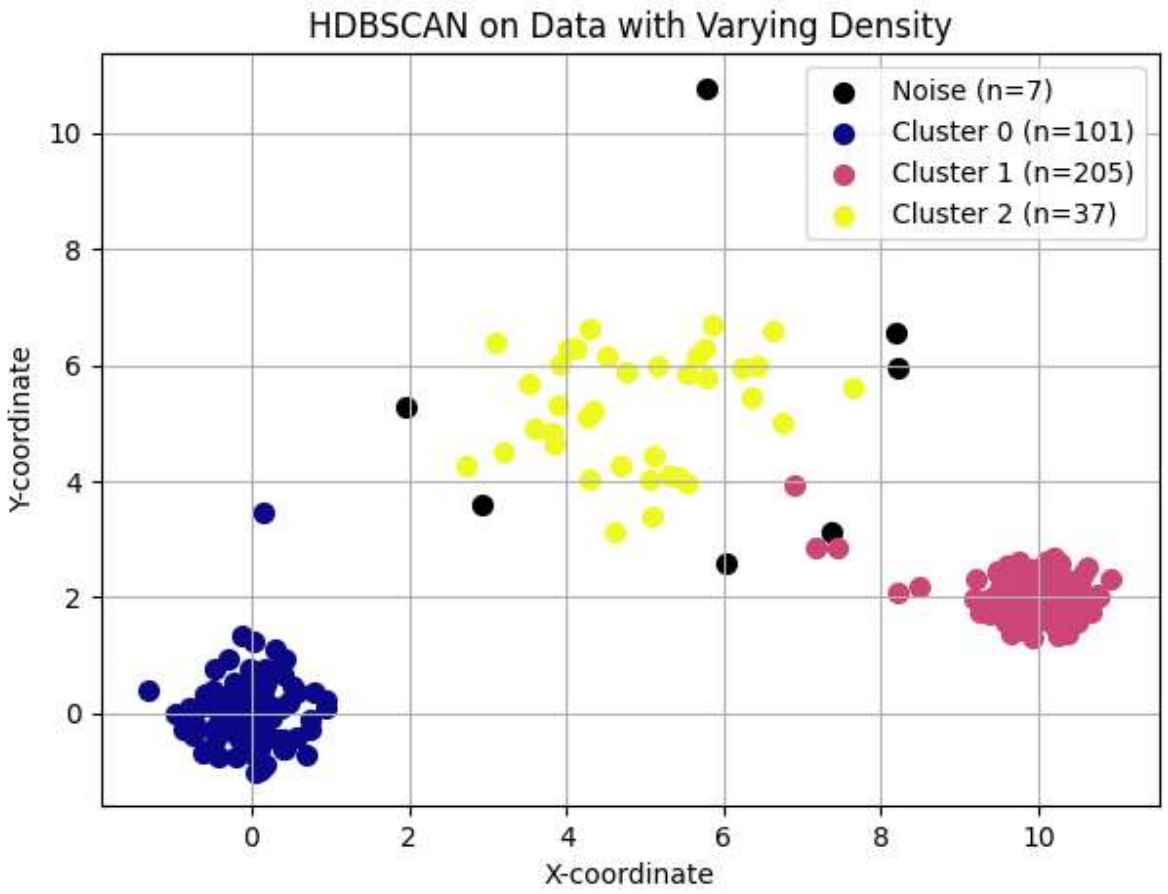


Figure 15: *HDBSCAN clustering on data with varying density (min_cluster_size = 15, min_samples = 5).*

Questions for Discussion

1. What are the advantages of DBSCAN over K-Means?

   DBSCAN does not require the number of clusters to be specified in advance and can discover clusters of arbitrary shape, such as non-spherical or irregular structures that K-Means cannot represent. It is also able to detect outliers and label them as noise.

2. What are the limitations of DBSCAN?

   DBSCAN is sensitive to its parameters ε (epsilon) and min_samples. Small changes in these values can significantly affect the resulting clusters. It also struggles with datasets containing clusters of varying density, since a single global ε cannot accommodate both dense and sparse regions simultaneously. In addition, DBSCAN's performance can degrade on very large or high-dimensional datasets because distance measures become less meaningful in such spaces.

3. Can DBSCAN identify noise in the Iris dataset?

   Yes. DBSCAN labels points that do not belong to any sufficiently dense region as noise (label = −1).

4. Would DBSCAN work well in high-dimensional datasets?

   Not very well. In high-dimensional spaces, the notion of "distance" becomes less informative due to the curse of dimensionality, making it difficult to define meaningful density thresholds. As a result, DBSCAN may either classify most points as noise or merge clusters incorrectly. Dimensionality reduction techniques such as PCA or t-SNE are often applied before using DBSCAN on high-dimensional data.

Conclusion

Through experiments on both synthetic and real-world datasets, DBSCAN was shown to effectively identify clusters of arbitrary shape without requiring the number of clusters to be specified in advance. Its ability to label low-density points as noise further distinguishes it from centroid-based algorithms such as K-Means, which assume spherical clusters and assign all data points to a group.

However, DBSCAN's performance was found to be highly dependent on the choice of its parameters, ε and min_samples, and it struggled when clusters exhibited varying densities. In contrast, HDBSCAN addressed this limitation by adaptively determining density thresholds, yielding more stable results and better separation of clusters with differing densities.

DBSCAN is most commonly applied in scenarios where data exhibit irregular shapes, variable densities, or contain noise that should not be forced into clusters. It is widely used in spatial data analysis, such as identifying geographic regions of interest or detecting hotspots in GPS data, as well as in anomaly and outlier detection in network security, manufacturing, and sensor data. DBSCAN is also applied in image processing, genomics, and astronomical data analysis, where natural groupings are complex and not easily captured by algorithms assuming spherical clusters or uniform densities.

Lab 3 - Hierarchial Clustering

Objective

This experiment aims to explore how hierarchical clustering groups data by progressively merging similar observations to form a tree-structured representation of clusters. The lab focuses on:

- Understanding the principles of agglomerative hierarchical clustering (the bottom-up approach), which is the most commonly used method.
- Visualizing how clusters are formed through a dendrogram, which reveals the order and distance at which points or groups are merged.
- Examining the effect of different linkage criteria (single, complete, average, and Ward) and how they influence cluster shapes and merging behavior.
- Applying hierarchical clustering to both synthetic 2D data and a real-world dataset (Iris), to perform dimensionality reduction using PCA for visualization.
- Investigating methods for extracting clusters from a dendrogram and comparing hierarchical clustering with K-Means and DBSCAN on the same dataset.

Theory

Hierarchical clustering is an unsupervised learning technique that groups data based on similarity while preserving the structure of how clusters are formed. Unlike partitioning algorithms such as K-Means, it does not require specifying the number of clusters in advance. Instead, hierarchical clustering organizes data into a multi-level hierarchy, often visualized as a tree-like structure called a dendrogram.

There are two main approaches:

1. **Agglomerative clustering (bottom-up):**
   This method starts by treating each data point as its own cluster. At each step, the two closest clusters are merged based on a chosen distance metric and linkage criterion. This process continues until all points form a single cluster. Agglomerative clustering is the most widely used approach and the one explored in this lab.
2. **Divisive clustering (top-down):**
   This approach begins with all data points in a single cluster and iteratively splits clusters into smaller ones. Although conceptually opposite to the agglomerative method, divisive clustering is less common due to its higher computational cost.

To decide which clusters to merge (or split), hierarchical clustering relies on a linkage criterion, which determines how distances between clusters are calculated. The most common linkage methods include:

1. **Single linkage:** Single linkage calculates the distance between two clusters by looking at the closest pair of points across them. This approach often creates long chain-like clusters because a single close connection can link two otherwise distant groups.
2. **Complete linkage:** Complete linkage measures the distance between the farthest points in each cluster. This method produces compact and well-separated clusters, but it can sometimes split clusters that lie close to each other.
3. **Average linkage:** Average linkage uses the mean distance between all pairs of points across the two clusters. This approach provides a balanced behaviour, avoiding the chaining effect of single linkage and the strict compactness of complete linkage.
4. **Ward linkage:** Ward linkage merges clusters in a way that causes the smallest increase in total within-cluster variance. It often creates the most compact and well-structured clusters, which makes it a common default choice in many applications.

Hierarchical clustering is most useful when the goal is not only to cluster data but also to understand the relationships between clusters, visualize how they form, and explore structure at multiple granularity levels. However, it can be computationally expensive for large datasets and may be sensitive to noise depending on the linkage method used.

Code

The DBSCAN clustering code can be found at the following GitHub link: Hierarchial_code

Synthetic Data and Clustering

To begin the experiment, a simple two-dimensional dataset was generated using the make_blobs function. The dataset contains 200 samples grouped around three well-separated cluster centers, each with a moderate standard deviation. This setup provides a clear structure that allows hierarchical clustering to reveal how points gradually merge into larger groups.

The scatter plot of the generated data shows three compact and visually distinct clusters. Each point represents an individual sample in the dataset. Because the clusters are well defined and do not overlap, this synthetic example offers an ideal starting point for observing how the hierarchical algorithm progressively builds the clustering hierarchy during the merging process.

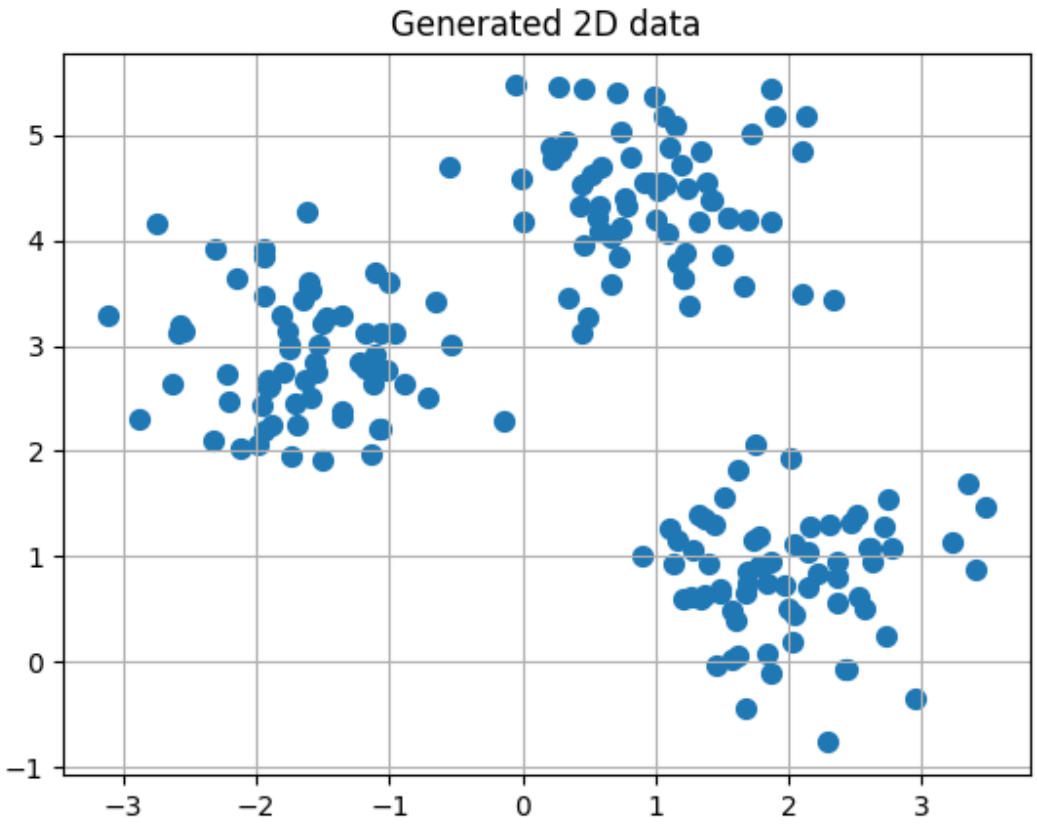


Figure 16. *Generated 2D synthetic dataset.*

Agglomerative Clustering with Dendrogram

After generating the synthetic dataset, the next step involves applying agglomerative hierarchical clustering and visualizing how the clusters merge over time. The algorithm begins with each data point treated as an individual cluster. At each iteration, it identifies the two clusters that lie closest to each other according to the chosen linkage method (Ward's method) and merges them. This repeated merging gradually builds the full hierarchical structure.

The dendrogram provides a visual summary of this merging process. Each horizontal line in the diagram represents a merge between two clusters, and the height of the line reflects the distance at which the merge occurs. Lower merges indicate pairs of points or clusters that lie close together, while higher merges correspond to groups that are more dissimilar. By examining the overall structure of the dendrogram, it becomes possible to see how the three main clusters form and at what distances they join.

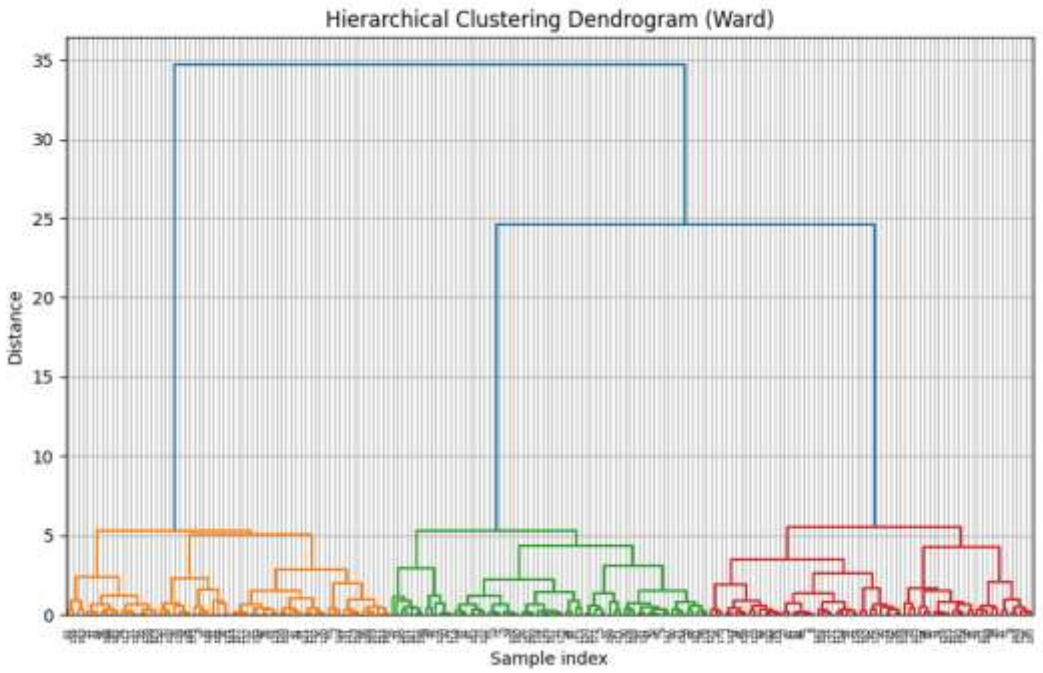


Figure 17. *Hierarchical clustering dendrogram (Ward linkage).*

Cluster Extraction

Once the hierarchical structure is built, the next step is to extract a specific number of clusters from the dendrogram. In this experiment, the agglomerative algorithm uses Ward linkage and forms three clusters, which matches the natural grouping in the synthetic dataset. By specifying n_clusters = 3, the algorithm stops the merging process at the appropriate level and assigns each data point to one of the resulting clusters.

The scatter shows the final clustering outcome. Each color represents one of the three clusters identified by the algorithm. The model clearly separates the data into the same groups visible in the original dataset, and the cluster boundaries align well with the natural structure. This result confirms that hierarchical clustering can accurately recover well-defined groups and provides a transparent merging path through the dendrogram.

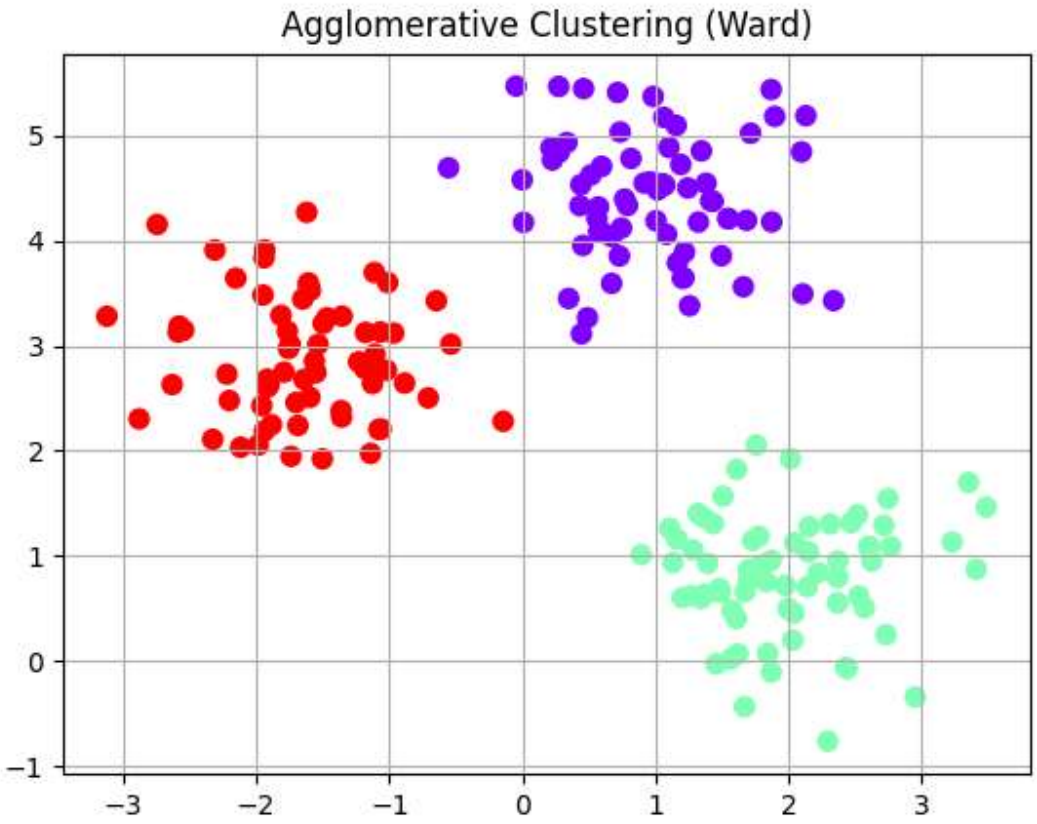


Figure 18. *Agglomerative clustering result using Ward linkage.*

Comparing Linkage Methods

Different linkage methods create noticeably different clustering results because each method uniquely calculates the distance between clusters. In this part of the lab, four linkage strategies (single, complete, average, and Ward) were applied to the same synthetic dataset from the earlier sections. Each method attempted to merge clusters based on similarity, but each one used a different definition of distance, which led to variations in cluster shapes and boundaries. These differences showed how strongly the choice of linkage influenced the structure of the final clustering result.

Figure 19 shows how the algorithm partitions the data under each linkage method. Single linkage often produces elongated, chain-like clusters because it merges groups as soon as any pair of points lies close together. Complete linkage behaves more conservatively by considering the farthest points between clusters, which results in compact and well-separated groups but can sometimes split clusters that sit close to one another. Average linkage balances these two extremes by merging clusters based on the mean distance between all point pairs across them. Ward linkage typically generates the most compact and stable clusters because it minimizes the growth of within-cluster variance at every step.

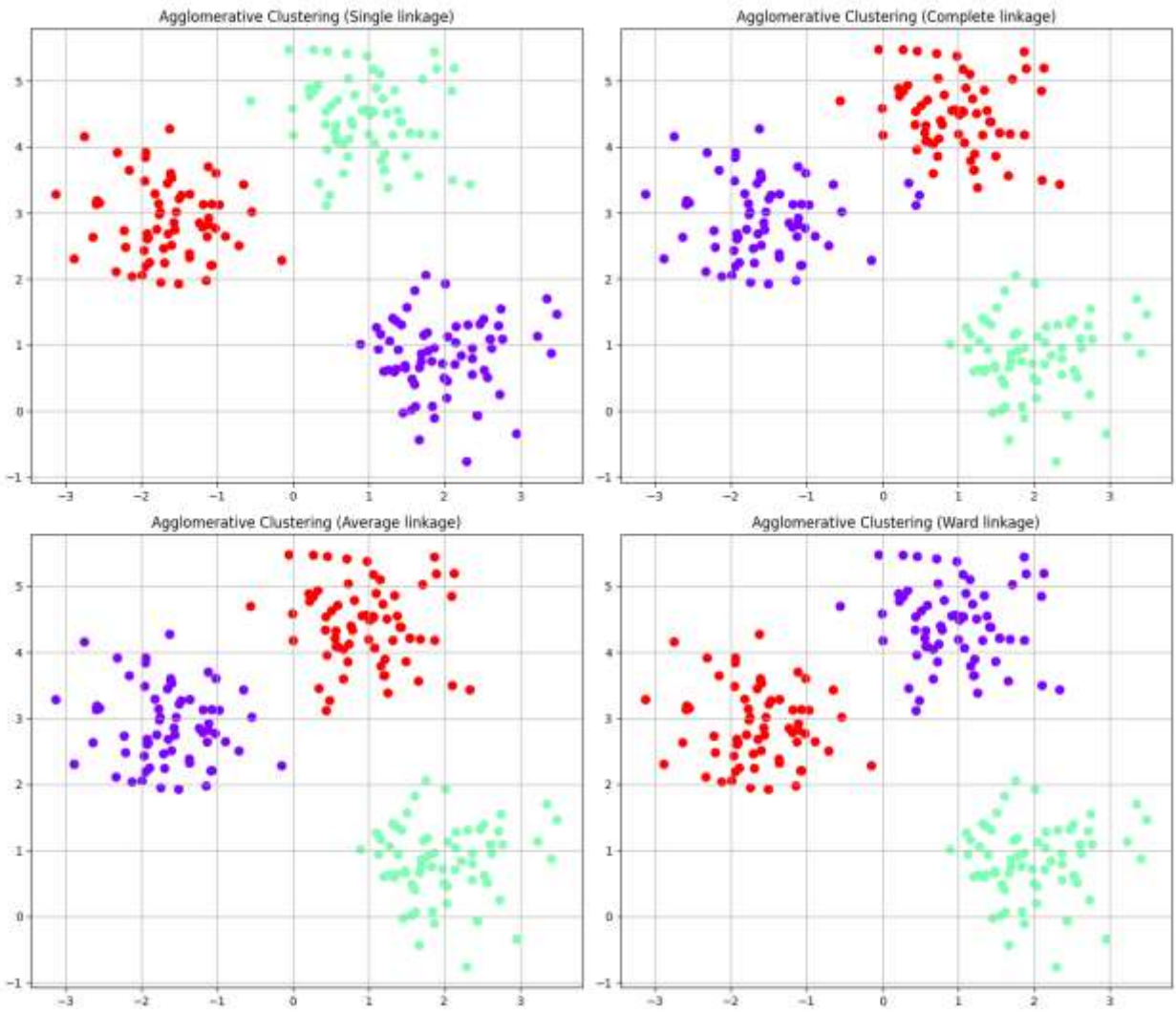


Figure 19. *Clustering results using single, complete, average, and Ward linkage methods.*

The dendrograms in Figure 20 further illustrate how each linkage method builds the cluster hierarchy. The height of the merges and the structure of the branches vary from method to method, showing how different distance interpretations influence the merging order. These differences demonstrate that the choice of linkage plays a significant role in shaping the final clustering structure, especially when the data contains subtle patterns or when cluster boundaries are not sharply defined.

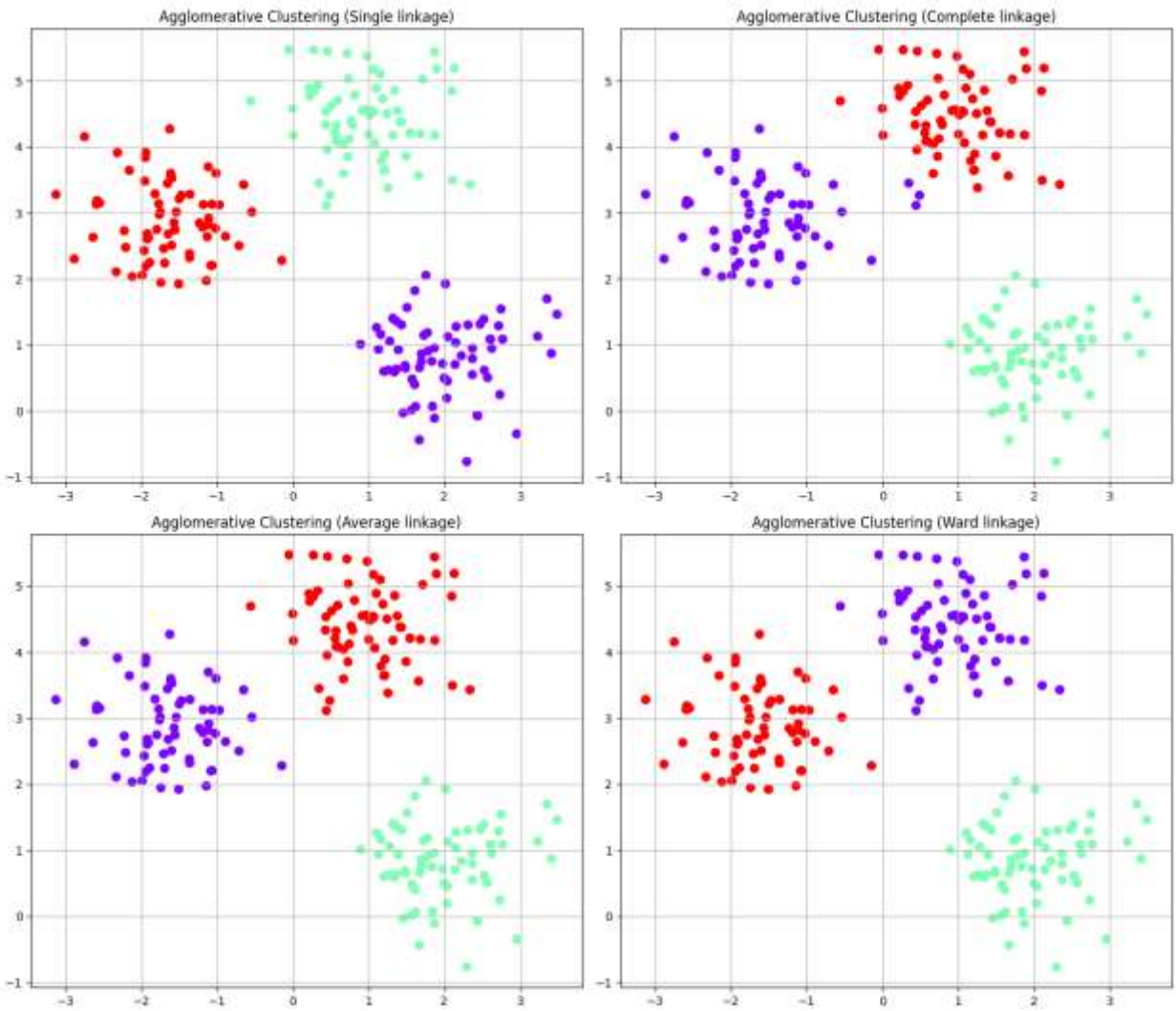


Figure 20. *Dendrograms for each linkage method.*

Iris Dataset (2D Projection with PCA)

The experiment used the Iris dataset to evaluate hierarchical clustering on real-world, multi-dimensional data. This dataset contains 150 flower samples with four numerical features that describe sepal and petal dimensions. Before running the clustering algorithm, the experiment scaled all features with Standard Scaler so that each variable contributed equally to the distance computations.

After scaling, the experiment applied Principal Component Analysis (PCA) and reduced the data to two principal components. This transformation preserved most of the variance in the dataset while providing a convenient two-dimensional representation for visualization. The clustering step then used agglomerative hierarchical clustering with Ward linkage and requested three clusters, which corresponds to the three known Iris species.

The figure below shows the clustering result projected onto the first two principal components. The three clusters appear as distinct groups in the PCA space, and their structure broadly matches the expected species separation. One cluster aligns clearly with Iris setosa, while the other two clusters overlap more, reflecting the known similarity between Iris versicolor and Iris virginica. This behavior agrees with observations from previous labs, where algorithms also found it harder to separate these two species.

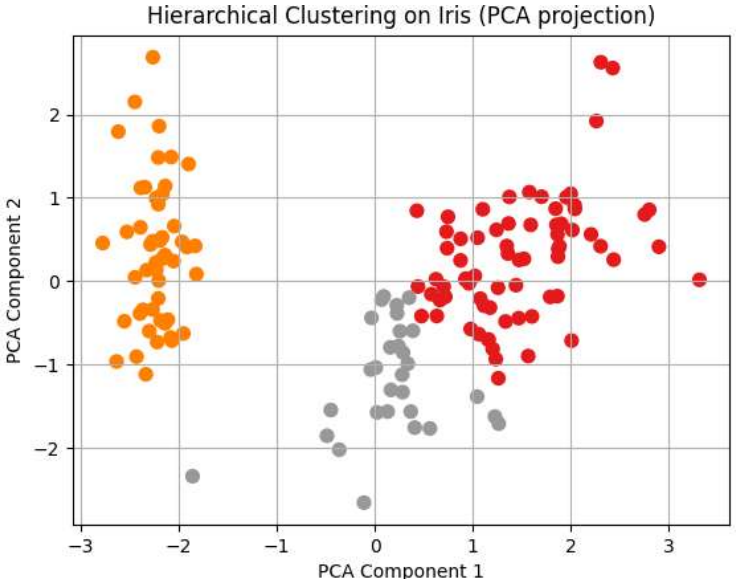


Figure 21. *Hierarchical clustering on the Iris dataset visualized in the first two principal components.*

The figure below presents the dendrogram obtained from the scaled Iris data using Ward linkage. The diagram reveals several meaningful merge levels and confirms the presence of three main groups, while also showing how individual samples join these groups at different distances. The dendrogram therefore adds an extra layer of interpretability beyond the 2D scatter plot.

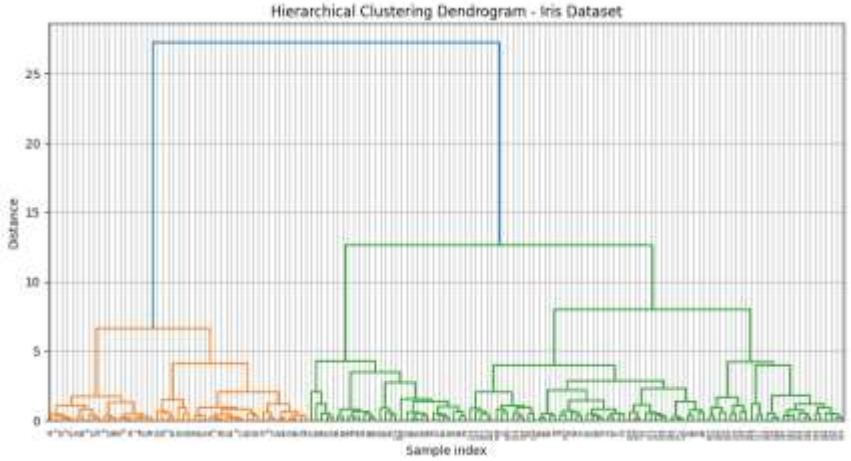


Figure 22. *Dendrogram for the Iris dataset using Ward linkage.*

Extracting Clusters Using Distance Thresholds

To explore the hierarchical structure in more detail, the experiment used the fcluster function to extract clusters by cutting the dendrogram at specific distances. This approach makes it possible to control the level of granularity in the clustering result. A smaller cut height produces more clusters by stopping the merging process early, while a larger cut height merges points into broader groups. The figure below shows the clusters obtained when the dendrogram was cut at a distance of 10. The algorithm returned three clusters, which aligns with the natural structure of the synthetic dataset.

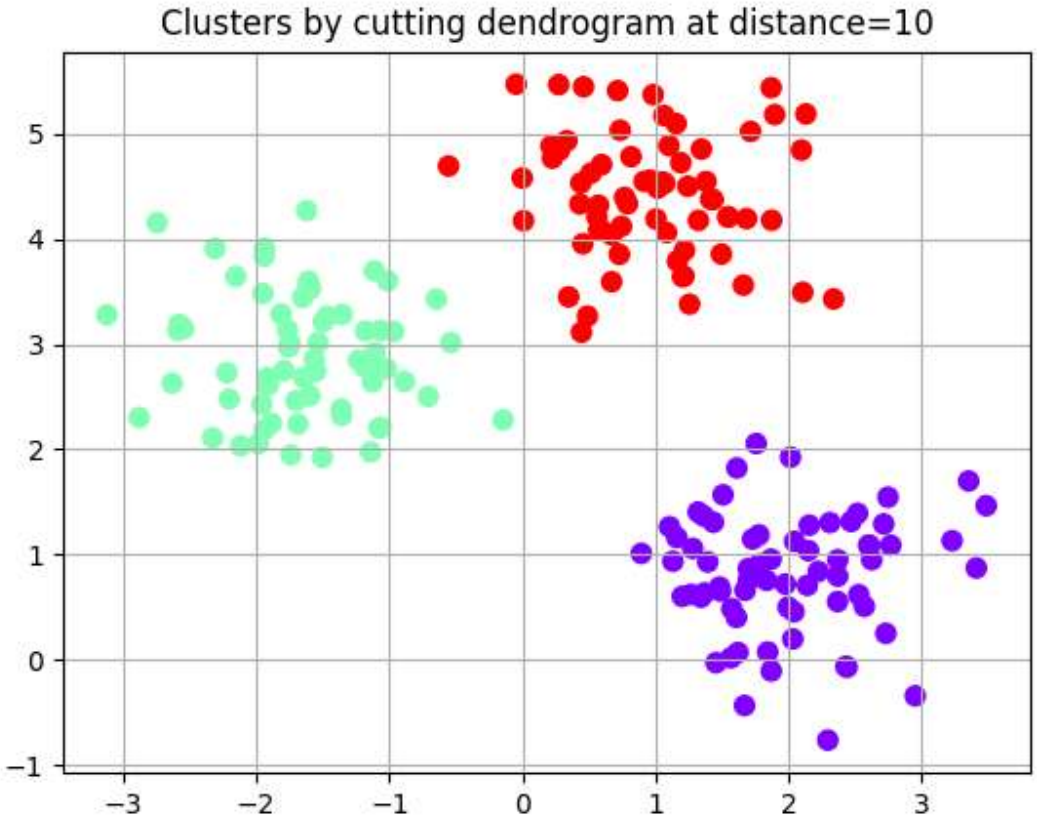


Figure 23. *Clusters extracted by cutting the dendrogram at a distance threshold of 10.*

Effect of Different Distance Thresholds on Clustering

To understand how the threshold influences the number of clusters, the experiment applied four distance values: 5, 10, 15, and 20. Each value produced a different number of clusters, ranging from several small groups at low thresholds to a nearly unified structure at high thresholds. These results show how hierarchical clustering reveals different layers of structure depending on where the dendrogram is cut. The figure below illustrates how the clustering result evolves across the four thresholds and highlights the sensitivity of the method to the choice of cut height.

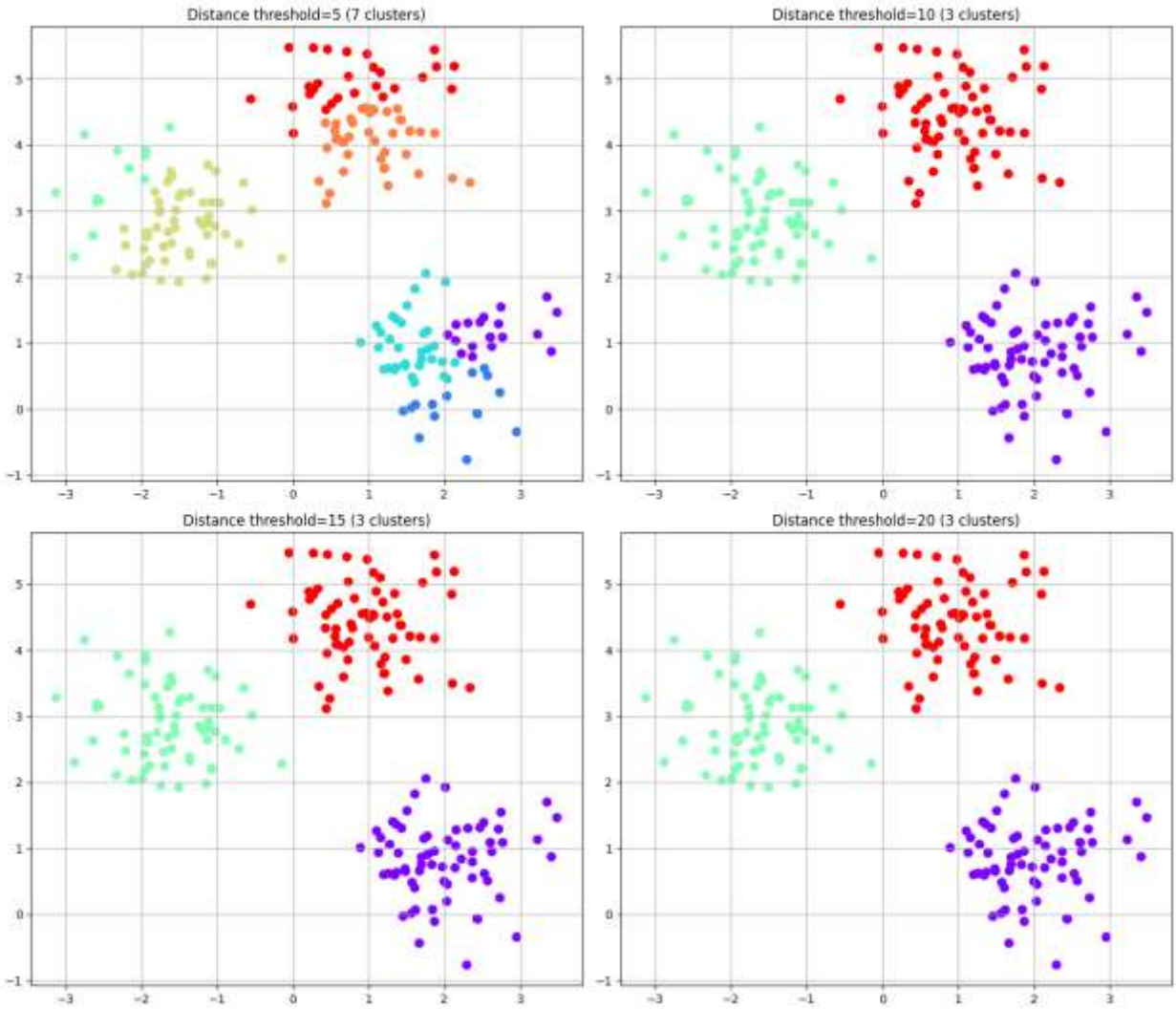


Figure 24. *Clustering results for distance thresholds of 5, 10, 15, and 20.*

Comparing Hierarchical Clustering with K-Means and DBSCAN

The comparison shows that each algorithm defines clusters differently and therefore produces distinct interpretations of the same dataset. K-Means works well when clusters are roughly spherical and similar in size, as it partitions the data using distances to fixed centroids. DBSCAN relies on local density, allowing it to capture arbitrary shapes and identify noise, but its performance depends strongly on the choice of parameters and can degrade when cluster densities vary. Hierarchical clustering takes a distance-based merging approach that preserves the relationships between points across multiple scales, enabling meaningful clusters to be selected directly from the dendrogram.

The side-by-side results show how each algorithm interprets the same dataset under their respective assumptions. K-Means separates the data into three compact, centroid-based clusters, which aligns well with the underlying blob structure. Hierarchical clustering with Ward linkage

produces a very similar partition, recovering the same three natural groups using a distance-based merging strategy. In contrast, DBSCAN groups all points into a single cluster and identifies no noise, indicating that the dataset does not contain strong density variations for DBSCAN to separate. This comparison illustrates how cluster geometry and density structure influence the outcome of each algorithm and why no single method behaves consistently across all datasets.

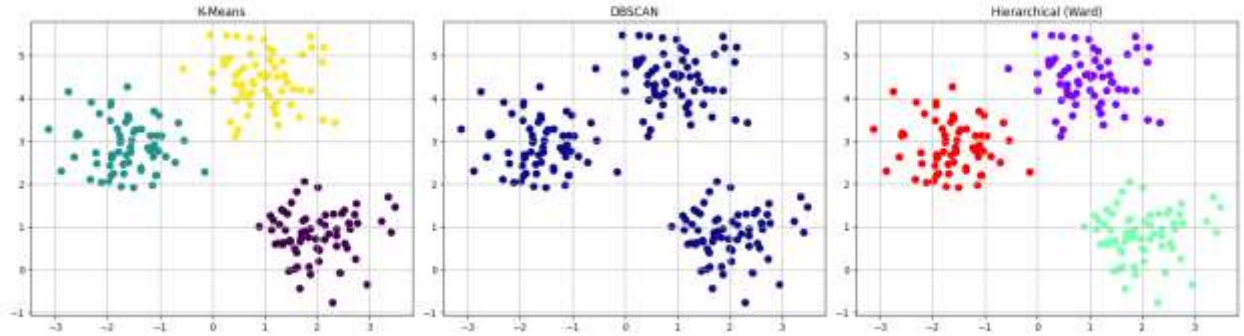


Figure 25. Comparison between K-Means, DBSCAN, and hierarchical clustering.

Questions for Discussion

1. What are the advantages of hierarchical clustering?
   Hierarchical clustering works well on small to medium-sized datasets because it does not require specifying the number of clusters in advance and provides a dendrogram that reveals the full merging history, allowing you to explore the data's structure at multiple levels of granularity with a level of interpretability that many other clustering methods cannot offer.
2. When is it better to use single or Ward linkage?
   Single linkage is most effective when the data contain elongated or irregular patterns that require a flexible chaining structure, whereas Ward linkage offers a stronger choice when the objective is to obtain compact, well-defined, and clearly separated clusters by controlling the growth of within-cluster variance.
3. What are the limitations compared to K-Means or DBSCAN?
   Hierarchical clustering becomes increasingly expensive on large datasets because it must compute and store a full distance matrix, and unlike DBSCAN it cannot naturally detect noise or adapt to variations in density, which makes it more sensitive to outliers and less practical than methods such as K-Means or DBSCAN when the data scale or density structure becomes complex.
4. How can we automatically determine the number of clusters?

A dendrogram allows you to estimate the number of clusters by choosing a horizontal cut where large vertical gaps reveal natural separations in the data, and this decision can be supported by techniques such as inconsistency coefficients or distance thresholds, which help identify a meaningful cut height without requiring prior knowledge of how many clusters should be extracted.

Conclusion

In conclusion, hierarchical clustering provides a powerful and interpretable framework for understanding the structure of data at multiple resolutions. Unlike K-Means, it does not require specifying the number of clusters beforehand, and unlike DBSCAN, it does not depend on density thresholds or struggle with datasets of mixed densities. Instead, it offers a complete hierarchical representation that can be explored visually through a dendrogram, making it especially valuable for exploratory data analysis and for domains where understanding relationships between clusters is as important as the clusters themselves.

However, hierarchical clustering also has limitations: it can be computationally expensive for large datasets, sensitive to noise, and influenced by the choice of linkage method. The experiments in this lab showed that Ward linkage produces compact, well-defined clusters, while other linkages such as single or complete may behave very differently depending on the data.

When compared to K-Means and DBSCAN, hierarchical clustering strikes a balance between flexibility and interpretability. K-Means remains efficient and effective for spherical, well-separated clusters; DBSCAN excels at discovering arbitrary shapes and identifying noise; and hierarchical clustering provides a multi-scale view that reveals how clusters emerge and merge. Understanding these differences is essential for selecting the most appropriate algorithm for a given dataset.